Hertie School

# Natural Language Processing

## Session 9: Neural Models III
## Quiz 3 · LSTMs · Attention · Transformers

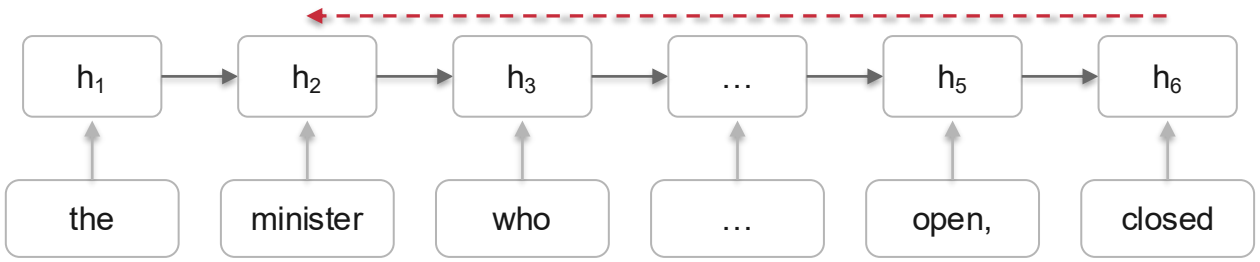
GRAD-E1282 · Fall Semester 2026
Luis Fernando Ramírez Ruiz · l.ruiz@hertie-school.org

# Quiz 3

30 minutes

# Where we left last class: three limits of the RNN

An RNN reads one token at a time and carries a single memory forward. That design costs three things.




The gradient from “closed” must travel back 11 steps to reach “minister”, halving at each: $0.5^{11} \approx 0.0005$ (Session 8, slide 19).

## 1 VANISHING GRADIENTS

The gradient is multiplied by the same weight at every step back: it vanishes (or explodes), so long links are never learned.

**Today:** LSTMs (slide 4), then attention: a direct line between any two tokens.

## 2 ONE TOKEN AT A TIME

$h_5$ waits for $h_4$, which waits for $h_3$. A 1,000-token article takes 1,000 steps, however many GPUs you have.

**Today:** all tokens are processed at once.

## 3 ONE-VECTOR BOTTLENECK

In the encoder–decoder (Session 8, slide 20), the whole input must squeeze into the last memory.

**Today:** every token keeps its own vector, and any later token may consult them all.

# The RNN's core problem: vanishing and exploding gradients

**BACKPROPAGATION THROUGH TIME (SESSION 8)**

The gradient (Session 8's "blame") travelling back from token t to token k is multiplied by the same recurrent weight w once per step: $w^{t-k}$.

Below 1 it vanishes; above 1 it explodes.

**FACTOR PER STEP, AFTER …**

| w | 10 tokens | 50 tokens | |
|---|---|---|---|
| 0.9 | 0.35 | 0.005 | vanishes |
| 1.1 | 2.6 | 117 | explodes |
| **1.0** | **1** | **1** | intact |

| | SIMPLE RNN · SESSION 8 | LSTM · HOCHREITER & SCHMIDHUBER, 1997 |
|---|---|---|
| **MEMORY** | rewritten at every token through $W_h$ and tanh | kept on a separate lane; gates erase or add |
| **GRADIENT, PER STEP BACK** | × $W_h$: vanishes or explodes | × the forget gate: near 1 when it must remember |
| **REMEMBERS, IN PRACTICE** | a few tokens | **dozens of tokens; 1,000+ steps in toy tasks** |

**Vanishing gradients are why a simple RNN forgets within a few tokens.** Clipping tames the explosions; the LSTM's memory lane fixes the vanishing.

# The parts of an LSTM cell

## STEP 0 · ONE INPUT FOR ALL FOUR

$h_{t-1}$ and $x_t$ → **$[h_{t-1}, x_t]$**

what the cell said last and the current token, joined into one vector (concatenated)

| vector | function | computed as |
|---|---|---|
| **$f_t$** forget | **σ → 0 to 1** | $\sigma(W_f\,[h_{t-1}, x_t] + b_f)$ |
| **$i_t$** input | **σ → 0 to 1** | $\sigma(W_i\,[h_{t-1}, x_t] + b_i)$ |
| **$\tilde{C}_t$** candidate | **tanh → −1 to 1** | $\tanh(W_c\,[h_{t-1}, x_t] + b_c)$ |
| **$o_t$** output | **σ → 0 to 1** | $\sigma(W_o\,[h_{t-1}, x_t] + b_o)$ |

## TWO STATES, CARRIED TO THE NEXT TOKEN

**$C_t$** cell state: long-term memory

**$h_t$** hidden state: what the cell shows to the next step and the classifier.

| acts on | what it decides | step |
|---|---|---|
| **$C_{t-1}$** | how much of the old memory to keep | 1 |
| **$\tilde{C}_t$** | how much of the candidate to write | 2 |
| **added to C** | what new content to write | 2 |
| **tanh($C_t$)** | how much of the memory to show as $h_t$ | 3 |

## INDEPENDENT, IN PARALLEL

Same input, own weights: no gate reads another gate. Trained together, they learn to cooperate. The order comes from how they are used (next slide).

## READS ≠ ACTS ON

A gate reads $[h_{t-1}, x_t]$ to decide, then acts on the memory. The forget gate does not forget $h_{t-1}$ or $x_t$: it scales $C_{t-1}$.

**Read $[h_{t-1}, x_t]$, decide with σ, act on the memory.** Only the candidate (tanh) proposes content; the three σ gates decide how much.

# How the parts interact: one step of an LSTM

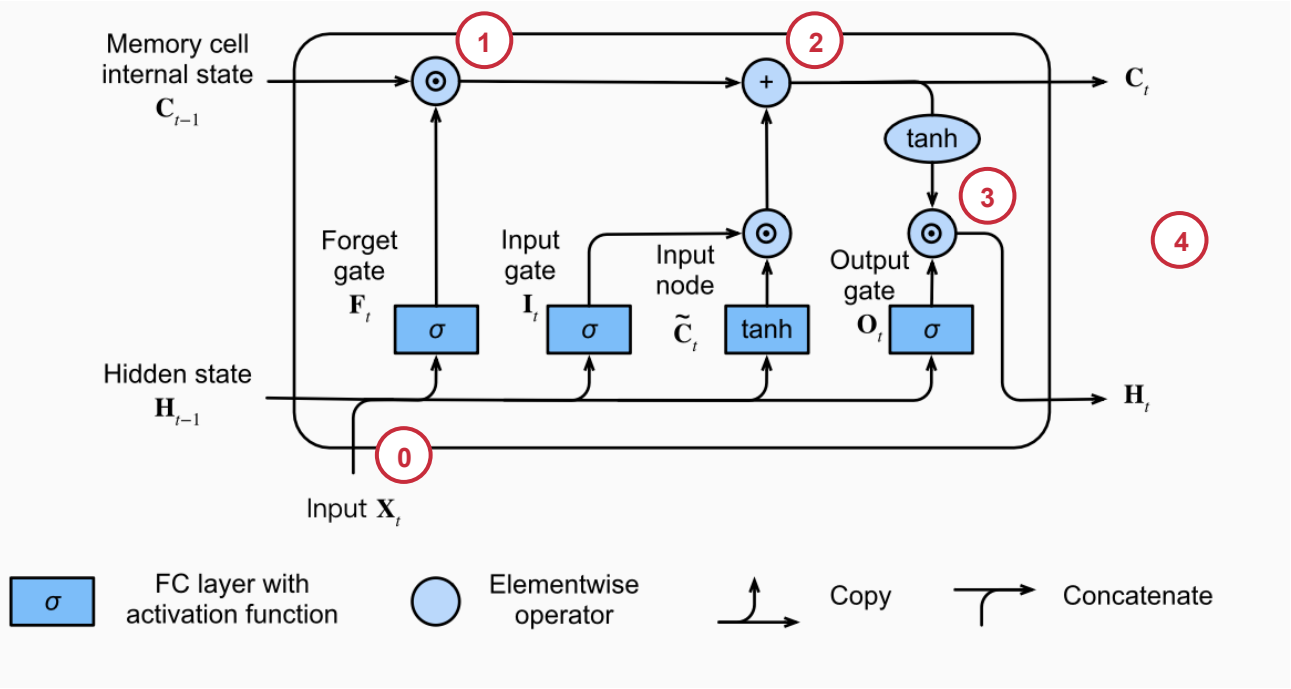



Figure: Zhang et al., Dive into Deep Learning, §10.1 (d2l.ai). The book writes F, I, O, H, X in capitals: the same f, i, o, h, x as slide 5. ⊙ = multiply slot by slot.

**0 JOIN THE INPUTS · BOTTOM LEFT**

$H_{t-1}$ and $X_t$ are concatenated; a copy goes to each of the four boxes. Each box has its own weights.

**1 FORGET · FIRST ⊙ ON THE TOP LINE**

$C_{t-1} \times f_t$

f (σ, 0 to 1) scales the old memory: 0.8 × 0.9 = 0.72

**2 WRITE · MIDDLE ⊙, THEN +**

$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t$

$\tilde{C}$ (tanh) proposes −0.6; i (σ) lets in half: 0.72 − 0.30 = 0.42

**3 OUTPUT · tanh, THEN RIGHT ⊙**

$h_t = o_t \times \tanh(C_t)$

o (σ) shows part of it: 1.0 × tanh(0.42) = 0.40

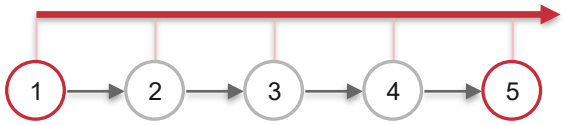
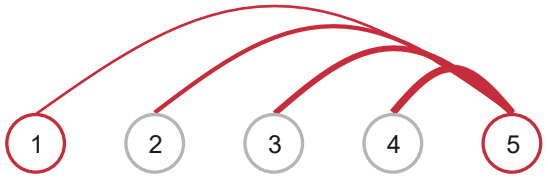
**4 PASS ON · RIGHT EDGE**

$C_t$ and $h_t$ leave the cell and enter the next token's step; $h_t$ also goes to the classifier.

**$C_t$ is what the cell remembers; $h_t$ is what it shows.** Going back along the belt multiplies only by $f_t$, so the gradient survives.

# From LSTMs to transformers: keep the ideas, drop the recurrence

Same goal as Session 8: read the tokens so far, predict the next one. What changes is how token 1 reaches token 5.

| | SIMPLE RNN · SESSION 8 | LSTM · SLIDES 5–6 | TRANSFORMER · AFTER THE BREAK |
|---|---|---|---|
| |  |  |  |
| | **4 hand-offs, each × $W_h$: fades** | **4 hand-offs on a gated lane: kept** | **1 step from every token, weighted** |
| MEMORY OF THE PAST | one vector, h | one vector C, plus h | **every earlier token's vector, kept** |
| RUNS IN PARALLEL | no: token after token | no: token after token | **yes: all tokens at once** |
| WEIGHTS COMPUTED FROM THE INPUT | none: only the fixed $W_h$ | gates: a σ valve on each memory slot | **attention: softmax weights over earlier tokens** |
| WHAT CARRIES OVER | The LSTM's two best ideas: weights computed from the input (gates → attention, slide 12) and a keep-and-add lane (cell state → residual connections inside each transformer block, Session 10). | | |

**A transformer keeps the LSTM's ideas and drops the recurrence:** every token reaches every earlier token directly.

# How did it all start? 75 years of predicting the next word

## BEFORE TRANSFORMERS

### 1948 — n-grams

Next-word statistics of English.

Session 3

### 1997 — LSTM

A memory lane that does not fade.

slides 4–6

### 2003 — Neural LM

A network predicts the next word, learning word vectors.

Session 8

### 2013 — Word2vec

Word vectors from predicting the neighbors.

Session 4

### 2014 — Encoder–decoder

Read a sentence into one vector, then write.

Session 8

### 2015 — Attention

The writer looks back at every input word.

Today and for the next sessions

## THE TRANSFORMER ERA

### 2017 — Transformer

Attention only, no recurrence. Built for translation.

### 2018 — GPT and BERT

Pre-train on raw text, then fine-tune. 117M and 340M parameters.

### 2020 — GPT-3

175B parameters. New tasks from a few examples in the prompt.

### 2022 — ChatGPT

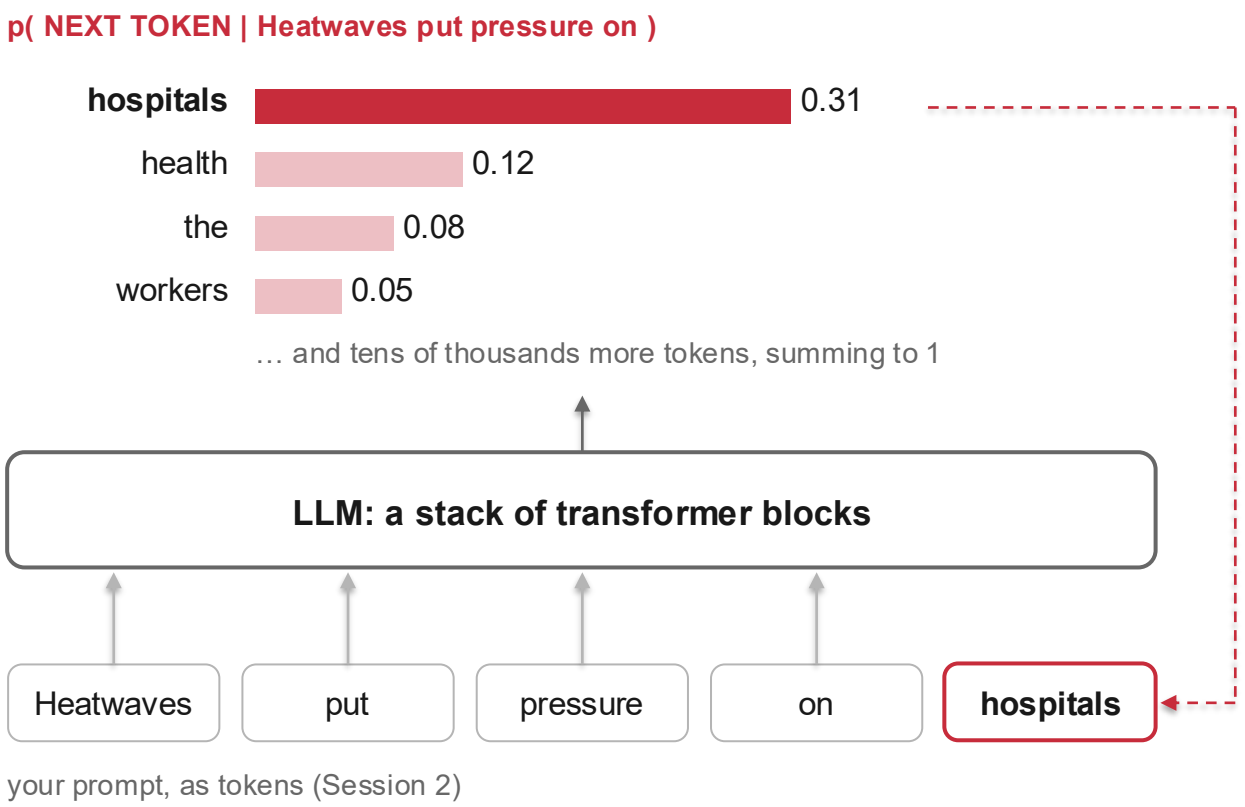
A GPT model tuned to follow instructions, with human feedback.

### 2023 — Open weights

Models you can download, fine-tune and audit.

**The goal never changed: predict the next word.** What changed is how much context a model can use, and how much text it learns from.

# Where we are now: LLMs predict the next token, on repeat




**1 READ**

The prompt is split into tokens; each becomes a vector.

**2 PREDICT**

The model gives every token in its vocabulary a probability of coming next.

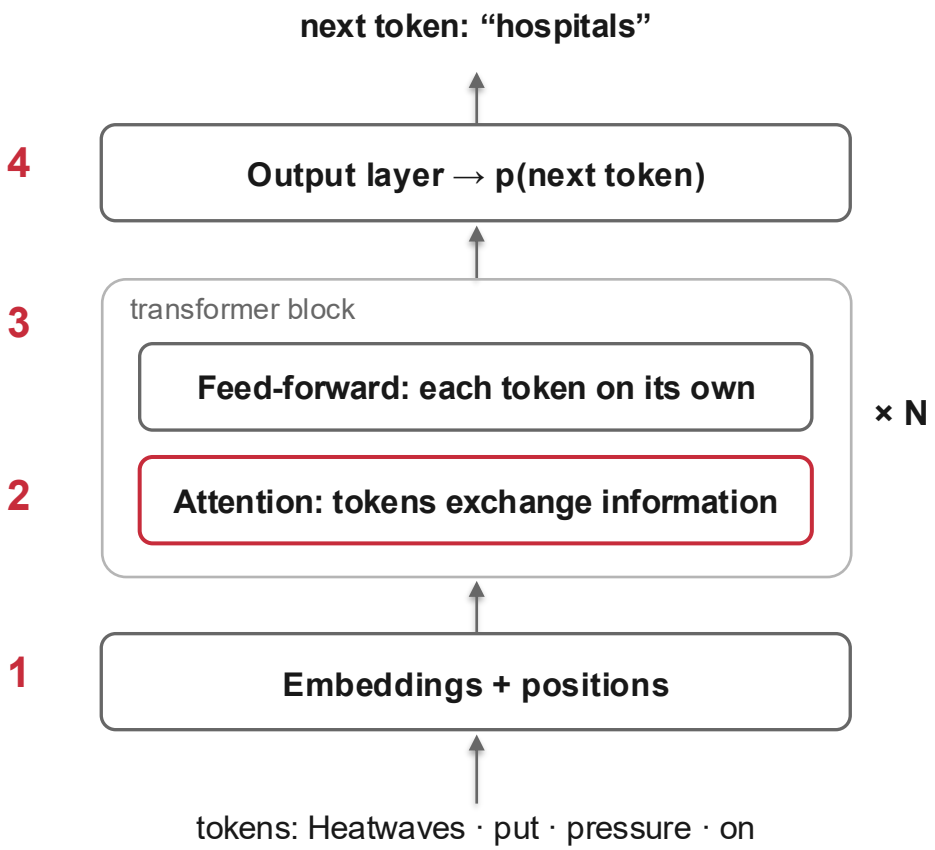
**3 APPEND, REPEAT**

One token is chosen and added to the input. A 300-word answer takes about 400 loops.

ChatGPT, Claude, Gemini, Llama: different data and tuning, the same loop.

**Session 8's language model, scaled up: predict the next token, append it, repeat.** Today we open the box.

# Why “transformer”? The architecture at a glance




### WHY “TRANSFORMER”?

Each block transforms every token’s vector into a richer one, using the other tokens. After 12, 32 or 96 blocks, “bank” no longer means just “bank”, but “bank, in this text”.

### WHY IT WON

No recurrence: all tokens are processed at once, so it trains fast on GPUs, and any two tokens are one step apart. “Attention is all you need” (Vaswani et al., 2017).

### WHAT YOU ALREADY KNOW

**1** embeddings (Session 4) · **3** feed-forward layers (Session 7) · **4** a softmax output (Session 5). Only **2**, attention, is new.

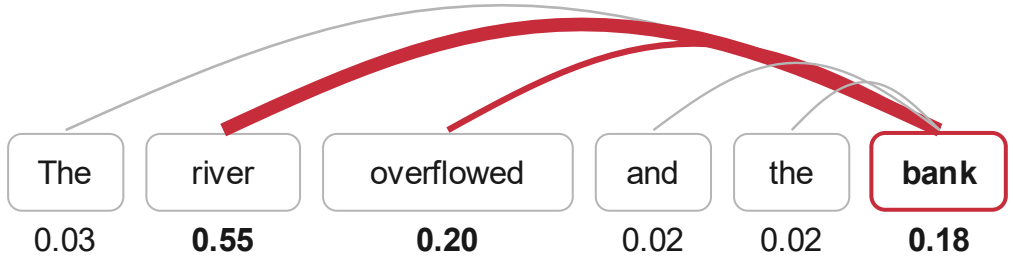
**A transformer turns static token vectors into contextual ones, block by block, then predicts.** After the break we study the one new part: attention.

# Session Break

When we return: attention, the idea at the heart of every transformer.

# What is attention?

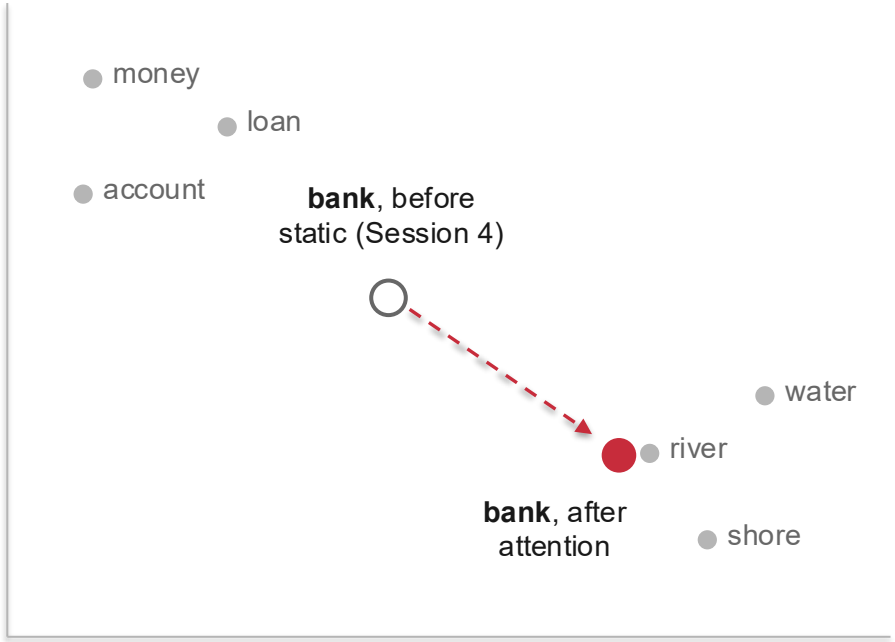
Attention gives each token a new vector: a weighted mix of the vectors of the tokens it can see, weighted by how relevant each one is.




**Which tokens can it see?** In GPT-style models, the ones before it, and itself. In BERT-style models, both sides.

**When?** In every layer, for every token at once.

**WHAT HAPPENS TO "BANK"**

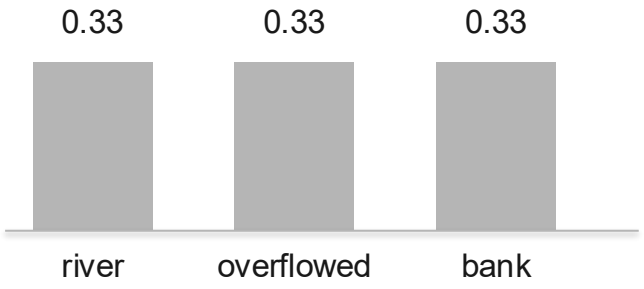



a 2-D cartoon; weights illustrative

**Static vector in, contextual vector out.** Session 8's RNN carried context forward step by step; attention fetches it directly.

# Attention is a weighted average

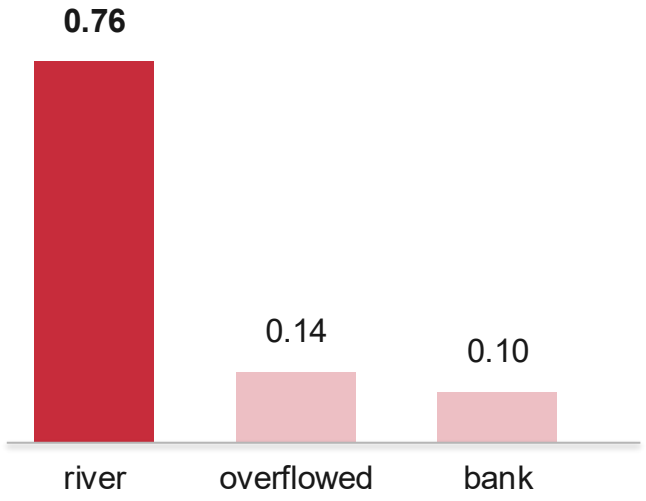
### SESSION 4 · PLAIN AVERAGE




$0.33\ v_{river} + 0.33\ v_{overflowed} + 0.33\ v_{bank}$

Equal weights, fixed once: the same vector whoever asks.

### TODAY · ATTENTION FOR "BANK"




$0.76\ v_{river} + 0.14\ v_{overflowed} + 0.10\ v_{bank}$

Weights computed for this token, in this context.

### THE SOFTMAX, BY HAND

| | score | $e^{score}$ | weight |
|---|---|---|---|
| river | 2.5 | 12.18 | **0.76** |
| overflowed | 0.8 | 2.23 | 0.14 |
| bank | 0.5 | 1.65 | 0.10 |
| sum | | 16.06 | 1.00 |

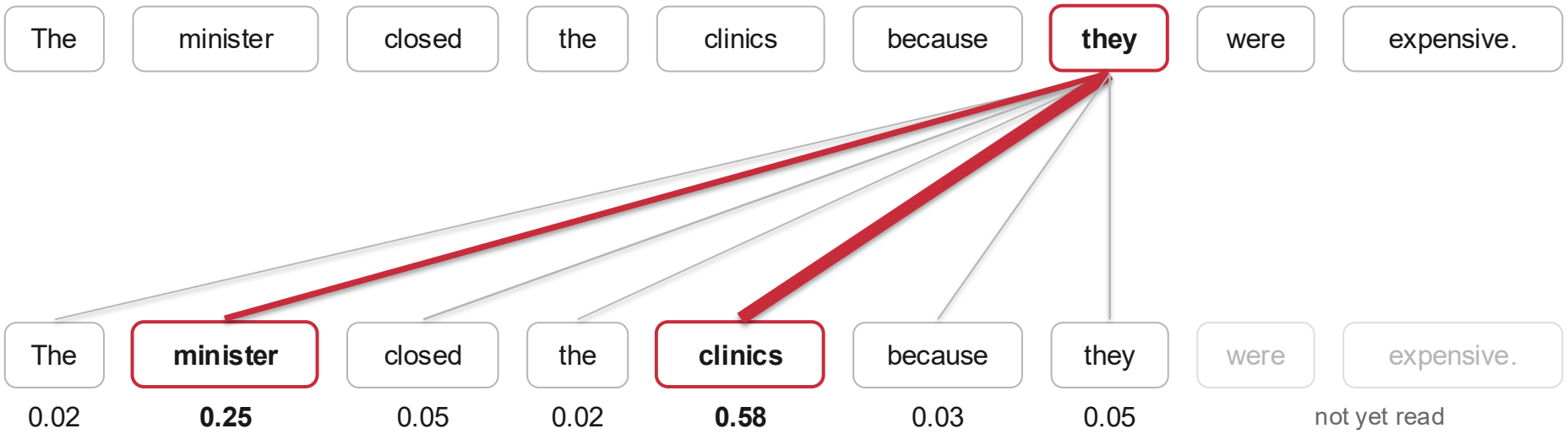
**score** = how similar "bank" is to each token: a dot product (Session 4).

**weight** = $e^{score}$ ÷ sum: Session 5's softmax.

**Attention is a weighted average whose weights are recomputed for every token that asks.** Session 4 averaged once, with equal weights.

# Self-attention: every token, at once, over the tokens before it

Every token runs the same weighted average at the same time, each over the tokens before it and itself.




**WHAT "THEY" GETS**

A new vector: 0.58 × clinics + 0.25 × minister + … It now carries what it refers to.

**WHAT EVERY OTHER TOKEN DOES**

The same, over the tokens before it: 9 tokens, 45 weights, all computed in one step.

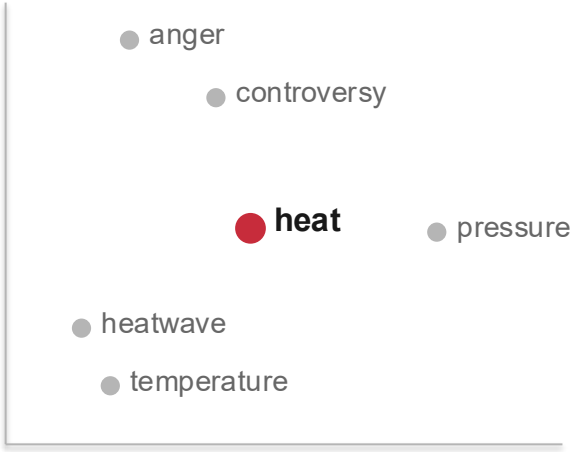
**Instead of carrying the past forward, each token fetches the context it needs, directly.** No memory has to survive the trip.

# Contextual embeddings: one word, a new vector in every sentence

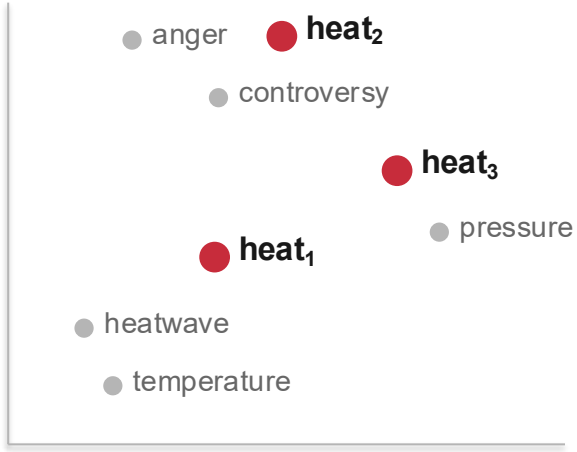
### THREE SENTENCES, ONE WORD

1. Hospitals filled up as the **heat** rose.
2. The debate produced more **heat** than light.
3. Police turned up the **heat** on the suspects.

### WORD2VEC · SESSION 4




### TRANSFORMER · TODAY




a 2-D cartoon; real vectors have hundreds of dimensions

### WORD2VEC

one vector per word type: a compromise between all its uses.

### RNN · SESSION 8

one per position, built step by step from the left.

### TRANSFORMER

one per position, fetched from its context, refined in every layer.

**These are the vectors BERTopic borrowed in Session 6:** its document vectors average contextual token vectors, not static ones.

# Where we are in the NLP pipeline




| Data acquisition | Text extraction, normalization | Representation / features | Basic models | Neural models | Transformers / evaluation | Transformers II | Agentic AI |
|---|---|---|---|---|---|---|---|
| Not covered | Session 2 | **Sessions 3–4** | Sessions 5–6 | SESSIONS 7–9 | Session 10 and on | Session 10 & 11 | Sessions 12 |

**LSTMs Attention Transformers: an introduction · next: inside the transformer, BERT and GPT (Session 10)**

**Today: from LSTMs to attention. Next: how a transformer is built around it, and how it is pre-trained.**